TRAINING CHESS BOTS WITH VARIOUS MACHINE LEARNING TECHNIQUES

by

BHAVYA SINGH
URN: 6839571

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Dr Joey Sik Chun Lam

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Bhavya Singh
May 2026

Abstract

Modern chess engines achieve superhuman play but depend on deep search trees that demand substantial computational resources. This dissertation asks whether a neural network can learn to play strong chess without any search at all. We train and compare three deep learning architectures, Convolutional Neural Networks (CNNs), Transformers, and Graph Neural Networks (GNNs), on millions of board positions labelled by Stockfish 16 at high depth. Each model receives an encoded board state and outputs a policy over legal moves, bypassing traditional tree search entirely.

We evaluate the resulting "search-less" models in two ways: first, by playing them against Stockfish at controlled depth limits to estimate their ELO ratings; second, by testing them on the Lichess Puzzle dataset to measure tactical accuracy across motifs such as forks, pins, and endgame conversions. Our comparison considers not only move accuracy but also inference speed and parameter efficiency, since a primary motivation for removing search is deployment on resource-constrained hardware.

The results show that each architecture captures different aspects of chess reasoning. CNNs excel at local pattern recognition, Transformers handle long-range piece coordination, and GNNs naturally represent the relational structure of legal moves. We discuss where each model fails, particularly in positions requiring deep calculation, and outline directions for closing the gap between search-based and search-free play.

Acknowledgements

Write any personal words of thanks here. Typically, this space is used to thank your supervisor for their guidance, as well as anyone else who has supported the completion of this dissertation, for example by discussing results and their interpretation or reviewing write ups. It is also usual to acknowledge any financial support received in relation to this work.

Contents

List of Figures

List of Tables

Glossary

N	Number of residual blocks in a network
F	Number of convolutional filters per layer
d	Embedding dimension (Transformer models)
h	Number of attention heads
L	Number of Transformer encoder layers
$\pi(a \mid s)$	Policy: probability of selecting move a in board state s
$v(s)$	Value: predicted game outcome from board state s
$\mathbf{X}$	Node feature matrix in GNN input
$\mathbf{A}$	Adjacency matrix representing piece connectivity or legal moves
cp	Centipawn; one hundredth of a pawn's value, used by engines to score positions
ELO	Rating system for measuring relative playing strength
ply	A half-move; one move by either White or Black

Abbreviations

CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
DTZ	Distance to Zeroing (tablebase metric)
ELO	Elo rating system (named after Arpad Elo)
FEN	Forsyth–Edwards Notation
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
MCTS	Monte Carlo Tree Search
MPS	Metal Performance Shaders (Apple GPU backend)
NNUE	Efficiently Updatable Neural Network
PGN	Portable Game Notation
PV	Principal Variation
SE	Squeeze-and-Excitation (block)
UCI	Universal Chess Interface
WDL	Win/Draw/Loss (evaluation format)
YAML	YAML Ain't Markup Language

Chapter 1

Introduction

1.1 Problem Statement

Chess has been a benchmark for Artificial Intelligence research for decades. It is a controlled, fully observable environment that exposes how well a machine can reason under combinatorial pressure. Early milestones such as IBM's Deep Blue [1] relied on handcrafted heuristics and "brute force" calculation. While effective, these systems were restrictive: they could not generalise beyond the specific rules their creators encoded. Today, neural-heavy architectures like AlphaZero, MuZero, and Leela Chess Zero dominate competitive computer chess. These engines have pushed ELO ratings past 3500, effectively ending the era of human-computer competition. But this strength comes at a cost. Modern engines rely on significant computational overhead and deep search trees to maintain their advantage.

1.2 Motivation

Current state-of-the-art engines are strong, but they are also "computationally greedy." Their performance comes from exploring millions of potential future states (search) before selecting a move.

This project pursues "Search-less Chess." By training Convolutional Neural Networks (CNNs), Transformers, and Graph Neural Networks (GNNs), we investigate whether a model can develop a "grandmaster's intuition," the ability to look at a board and immediately identify the best move without a search algorithm. Removing the search would reduce move latency and could reveal how machine learning architectures internalise spatial and strategic relationships.

1.3 The Challenge of Chess

Chess is a game of perfect information, yet it remains computationally "unsolved" because of its strategic depth. A player must balance tactical immediate threats (checks, captures) with abstract long-term goals (piece mobility, pawn structure, and king safety).

1.3.1 Computational Complexity

The complexity of chess is often illustrated by Shannon's Number, which estimates the game tree complexity at approximately 10^{120}. Even the state space, the number of possible legal positions, is estimated at 10^{40}. Because the number of possible variations grows exponentially with every half-move (ply), brute-force calculation is physically impossible. This requires an approach that either prunes the search space or, as we propose, bypasses it entirely through high-fidelity policy prediction.

1.3.2 Failure of Naive Approaches

Simple heuristics (e.g., assigning a point value to pieces) fail because chess is highly contextual; a Queen is worthless if the King is in an inescapable checkmate. Naive machine learning models regularly fall into "greedy" play, capturing material while ignoring a looming positional collapse. Without the "look-ahead" provided by search, a model must be considerably more sophisticated enough to recognise the fine hints that separate a winning position from a losing one.

1.4 Critique of Existing Solutions

Engines like Stockfish 18 are strong but operate as 'black boxes' of search optimisation. They require significant CPU/GPU resources to reach their peak ELO. Even hybrid models like Leela Chess Zero, which use neural networks for evaluation, still rely on Monte Carlo Tree Search (MCTS) [2] to validate their "hunches."

1.5 Overview of My Approach

1.5.1 Machine Learning Architectures

This project compares three architectures to determine which best captures the tactical and strategic-level patterns of chess without search.

- **CNNs:** Treat the board as a 2D image, capturing local spatial patterns and piece interactions.

- **Transformers:** Use self-attention mechanisms to model long-range dependencies across the board (e.g., a Bishop's influence from a8 to h1).
- **GNNs:** Represent the board as a dynamic graph where pieces are nodes and their legal moves are edges, emphasising the relational geometry of the game.

We train these models on the Lichess Stockfish dataset, effectively distilling the knowledge of a 3500-ELO engine into our search-less architectures.

1.5.2 Comparison Methodology

To validate our approach, we benchmark our models against Stockfish at varying "depth" constraints. This lets us simulate different ELO ranges and see where a search-less model begins to falter compared to a searching engine. We also use the Lichess Puzzle dataset to test the models on specific tactical motifs, providing a granular view of their "chess IQ" in areas such as forks, pins, and endgame transitions.

1.6 Summary of Key Results

1.7 Scope and Limitations

The goal of this research is not to compete with the world's strongest engines, but to push the ceiling of what is feasible within the "search-less" paradigm. We acknowledge that without search, our models could struggle with "horizon effects," where a tactic requires deep calculation beyond the model's immediate pattern recognition. Highly technical endgames (e.g., Rook and Bishop vs Rook) may also remain difficult for a purely intuitive model.

1.8 Aims and Objectives

The main goal is to engineer and evaluate deep learning models (CNN, Transformer, GNN) capable of achieving a 2000 ELO rating without search algorithms. Success is measured by:

1. Generalisation: whether the models have "learned" chess principles rather than memorised positions, verified via performance on an unseen puzzle set which have an ELO rating attached to it by solving a certain number of puzzles and fitting a curve on the puzzle solve percentage, we can estimate the ELO of the model on the puzzles.

2. Architectural comparison: which architecture (spatial, relational, or sequential) is best suited for the geometry of chess.

1.9 Report Structure

Chapter 2 surveys the history of chess AI and the development of deep learning in games. Chapter 3 details our methodology, covering data curation, normalisation, and the hyperparameters of each architecture. Chapter 4 presents the experimental results, including ELO benchmarking and tactical performance. Chapter 5 discusses the consequences of these findings for lightweight, search-free AI.

Chapter 2

Literature Review

2.1 Classical Chess Engines

Early chess engines relied on the minimax algorithm [3], combined with alpha-beta pruning, to efficiently search the game tree.

These engines used handcrafted evaluation functions that scored board positions based on material count, piece activity, king safety, and other heuristics. Deep Blue [1] is the canonical example.

Even though effective, these approaches were limited in adaptiveness and required extensive domain knowledge to tune. Stockfish, the strongest open-source chess engine, illustrates this classical approach with its highly optimised search algorithms and the evaluation function.

(Stockfish before 2020 was based on handcrafted features and alpha-beta search, but post-2020 it transitioned to a hybrid NNUE + alpha-beta architecture, where the evaluation function is a neural network trained on handcrafted features.)

2.2 Probabilistic Search Methods

Monte Carlo Tree Search (MCTS) [2] [4] is a probabilistic search method that builds a search tree by randomly sampling the game space, exploring promising moves without a detailed search. In complex game states with vast search spaces, MCTS has clear advantages over traditional minimax. Leela Chess Zero, for instance, combines MCTS with neural network evaluations.

MCTS also adapts its search strategy based on prior simulation outcomes, making it more flexible in dynamic game environments. This flexibility goes beyond chess: MCTS is central to AlphaGo's [5] Go play and to several general game-playing systems.

2.3 The Deep Learning Revolution

AlphaZero [5], developed by DeepMind, combined deep learning with self-play reinforcement learning and defeated Stockfish in a 2017 match. Its architecture has two components: a policy network that predicts move probabilities and a value network that estimates position outcomes. Through self-play alone, with no human game data, AlphaGo surpassed human-level play in chess, shogi, and Go.

Leela Chess Zero replicates AlphaGo's design and training methodology as an open-source project, and is now competitive with Stockfish at top-level play. The wider impact of AlphaZero's approach has been a shift toward neural network architectures [6] and self-play training in chess engine research.

2.4 Supervised Learning Approaches

DeepChess [7] uses a two-stage deep neural network: a Pos2Vec autoencoder pre-trained unsupervised on 640K human games from the CCRL database, followed by a supervised comparison network that takes two positions as input and outputs which is more favourable. The system never assigns numerical evaluations; it directly learns to rank positions, achieving grandmaster-level play without handcrafted heuristics.

Giraffe [8] trains a neural network evaluation function using temporal-difference learning via self-play, with minimal hand-crafted input features. Despite receiving no explicit human chess knowledge, the resulting engine plays at approximately FIDE International Master level (top 2.2% of rated players), comparable to the strongest handcrafted engines of the time. The key difference from DeepChess is that Giraffe uses self-play rather than human game databases, and TD-learning rather than supervised pairwise classification.

2.5 Recent Innovations

Graph representations of chess positions [9] [10] aim to capture the game's relational structure more directly than grid-based encodings. Rigaux and Kashima (2024) [9] proposed a graph-based neural network architecture that models interactions between pieces and their board positions, achieving improved evaluation accuracy and strategic insight compared to grid baselines.

Explainable AI (XAI) [11] [6] in chess is a newer research direction. Erik and Shreyas (2024) developed methods to visualise the influence of individual pieces and moves on engine evaluations, making the decision-making process of chess engines more transparent to both players and researchers.

AlphaGateau (Rigaux and Kashima) paper [9] demonstrates that while using fewer parameters and compute time compared to AlphaZero thy were able to reach an ELO of 2105 ± 42 with just 131k total frames which

were derived from 256 games, which when compared to a similarly sized AlphaZero model with 5 layers was only able to achieve an ELO of 667 ± 38 in 5x5 chess. Finetuning the AlphaGateau model on 8x8 chess improves the accuracy from 807 ± 46 to 1876 ± 47, which suggests that it was able to learn general chess rules from 5x5 and apply them to 8x8 with some success.

In the paper learned look-ahead in a chess playing neural network (Erik and Shreyas) [6], they were able to demonstrate that neural networks learn to look ahead, which helps them achieve higher ELO numbers despite not having the ability to use tree based search methods, which was later also shown by Google DeepMind team with their searchless chess paper [12].

2.6 Game AI Frameworks

The python-chess [13] library provides move generation and validation, PGN parsing and writing, Polyglot opening book reading, Syzygy/Gaviota tablebase probing, and full UCI [14]/XBoard engine communication. It is the standard Python interface for working with chess engines programmatically.

2.7 Web-Based Chess Platforms

2.7.1 Existing Interactive Chess Platforms

Lichess, developed by Thibault Duplessis, is an open-source chess server [15] written in Scala/Play with an asynchronous architecture using Akka actors, MongoDB for game storage (800M+ games), Elasticsearch indexing, and nginx proxying both HTTP and WebSocket connections. Real-time move streaming is handled via WebSockets.

2.7.2 Bot-vs-Bot Comparison Platforms

The Top Chess Engine Championship (TCEC) [16] is the most prominent example of a spectator-oriented bot-arena platform. The Chess Programming Wiki documents TCEC's format and various tournament manager implementations.

2.7.3 Human-vs-Bot Interfaces

The Lichess Board API [17] endpoints (boardGameStream, boardGameMove) are the reference implementation for streaming live moves between a bot and a human player in real time, using REST with NDJSON streaming.

2.7.4 Web Technologies for Real-Time Games

The WebSocket Protocol (IETF RFC 6455, 2011) [18] is the standard for full-duplex communication over a single TCP connection and underlies the game communication layer in this project.

The choice of Python and Svelte [19] for the backend and frontend, respectively, is well-suited to this project. Python allows us to really quickly load models without intermediate layers to run the actual models, while Svelte's reactive framework delivers efficient UI updates with minimal overhead.

2.8 Critical Analysis

Three architectural families dominate current research on chess AI: CNNs [7], transformers [20] [12], and graph neural networks [9] [10]. Each has particular strengths and weaknesses that motivate the comparative approach taken in this project. Stockfish post 2020 confirms that CNNs remain competitive in hybrid NNUE + α-β deployments, which are the most computationally efficient at inference time. However, CNNs have a structural limitation: a 3×3 kernel sees only adjacent squares, so many stacked layers are needed to propagate information across the board. This is a poor match for chess, where a single piece (e.g., a rook or queen) can instantly influence the entire board.

Transformers address this receptive field problem through global self-attention. Chessformer (CF-240M) [20] achieves 2347 Elo and 93.5% puzzle accuracy at 30× fewer FLOPs than comparable AlphaGo-scale models. The finding is that positional encoding is the decisive design choice: absolute, relative, and dynamic encodings produce markedly different results. However, quadratic attention complexity beyond 64 tokens, while manageable, is costly at scale compared to CNN inference, and transformers require careful positional encoding, since absolute embeddings fail to capture board geometry, and naive relative encodings lose piece-type information.

Daniel and Philip (2024) [20] and Google DeepMind (2024) [12] train a large transformer on Lichess games annotated with Stockfish. This is architecturally the closest existing work to the pipeline in this project, covering action-value and state-value heads and the tradeoffs between square-based and piece-based tokenisation strategies. The critical gap is that they evaluate a single architecture; this project systematically compares Square and Piece Transformers under identical conditions.

Graph neural networks offer a third alternative. AlphaGateau [9] (GAT + GATEAU edge-feature layer) outperforms ResNet-based AlphaGo by an order of magnitude in sample efficiency on a 5×5 chess variant, showing that the graph representations encode relational structure (legal moves as edges) that grids cannot be expressed compactly. Yet Rigaux and Kashima [9] train only on a 5×5 variant and fine-tune to 8×8; no published work presents a full GCN/GAT chess engine trained end-to-end on standard chess from scratch, and tested against

CNN and Transformer baselines on the same dataset. This is the most direct gap this project fills.

2.9 Summary

Two findings from the literature directly shape the methodology of this project. First, representation matters more than depth. Chessformer [20] shows that positional encoding is the decisive variable for transformers, while Rigaux and Kashima [9] show that graph edges (encoding legal moves) capture the relational structure that grid-based representations miss. A multi-architecture comparison is the natural empirical test of this claim. Second, supervised Stockfish annotation is a viable and scalable training signal: Chessformer, Searchless Chess [12] all validate this pipeline, but none investigate PV unrolling for position diversity. Where we take the Lichess dataset and the top Stockfish line and recursively apply the moves suggested in the line and hence giving us 10 different positions to train from one sample.

The main reason to use three architectures is that chess is a complex domain with multiple interacting patterns. CNNs are good at local spatial patterns, transformers excel at long-range dependencies, and GNNs capture relational structure. By comparing all three on the same dataset and evaluation benchmarks, we can identify which architectue is most effective for search-less chess.

The central gap in the existing literature is the absence of a fair cross-architecture benchmark. Every existing paper evaluates one architecture in isolation. This project's controlled comparison on identical Lichess/Stockfish data with PV-unrolled positions addresses that gap directly.

Chapter 3

Methodology and Design

3.1 Requirements Analysis

The framework had to support a fair, controlled comparison of deep learning architectures for searchless chess. The following requirements shaped its design.

3.1.1 Functional Requirements

1. The framework must train and run inference across at least three neural network families — CNNs, Transformers, and GNNs — under identical data and evaluation conditions.
2. Any backbone (e.g., ResNet, GAT) must compose with any output head (Policy, Value, or Dual) without code duplication.
3. The system must ingest engine-annotated positions from Parquet files and train models using hard policy labels alongside centipawn-derived value targets.
4. A multi-stage data preparation pipeline must handle over one billion positions, including deduplication and PV line unrolling.
5. An automated benchmarking pipeline must pit trained models against Stockfish at configurable depths, recording per-move evaluations and exporting PGN files for post-hoc analysis.
6. A common agent interface must let any model, engine, or random baseline participate in games interchangeably.

3.1.2 Non-Functional Requirements

1. Training must use mixed-precision arithmetic (float16) and `torch.compile` where applicable to maximise GPU throughput.
2. The data pipeline must stream from disk, one row group at a time, keeping RAM usage bounded regardless of dataset size.
3. Each subsystem (encoders, models, training loops, agents) must reside in its own package with clearly defined interfaces, so that it can be developed and tested independently.
4. All hyperparameters must be specified in a single YAML configuration file, and training checkpoints must be restorable for exact experiment replay.
5. Adding a new backbone or encoder should mean implementing a single abstract class — nothing more. Existing training and evaluation code should need zero changes.

3.2 Framework Architecture

The framework is organised into five principal modules, each owning a distinct concern.

3.2.1 Chess Environment Module

The chess environment module wraps the `python-chess` library to handle board management and move encoding. A statically generated lookup table maps every possible UCI move string (including promotions) to a unique integer index; this table has 4544 entries and acts as the universal action space for all policy heads. We then monkey-patch the lookup table with moves related to castling due to the fact that dataset utilizes chess960 notation, whereas `python-chess` expects standard notation.

Three specialised state encoders translate a `chess.Board` object into the tensor format each architecture family expects:

- **CNNEncoder:** In its initial form, the encoder produces an $18 \times 8 \times 8$ tensor with 12 bitboard planes (six piece types per colour), four constant planes for castling rights, one plane for the en passant square, and one plane encoding the side to move as $+1$ (White) or -1 (Black). The board is always encoded from the current player's perspective — ranks and files are flipped when Black is to move.
- **GNNEncoder:** Constructs a graph with 64 nodes, one per square. Each node carries a 14-dimensional feature vector: a 13-element one-hot piece identifier and a colour scalar. Edges come from two sources —

physical adjacency (King move patterns) and tactical relations (`board.attackers`) — and carry three-dimensional attributes: normalised Manhattan distance, a binary ray-tracing flag for unblocked lines of influence, and a binary legality flag. Reverse edges are added to preserve structural symmetry while keeping directional legality features intact.

- **TransformerEncoder:** Supports two tokenisation schemes. The **SquareTokenizer** emits a fixed-length sequence of 64 tokens (one per square) with a 13-class vocabulary (empty plus twelve piece types), normalised rank/file coordinates, and full attention. The **PieceTokenizer** emits a variable-length sequence of up to 32 tokens (one per piece), ordered by colour (current player's pieces first), with each token carrying its piece type and square position. Padding masks stop empty slots from contributing to attention.

All three encoders inherit from an abstract `StateEncoder` base class that mandates `encode()`, `get_input_shape()`, and `name` methods. The factory layer uses these to pick the correct encoder at runtime based on the chosen backbone.

3.2.2 AI Agent Module

The agent module defines a common interface through the abstract `ChessAgent` class, which requires every agent to implement `get_move(board, time_limit)` and a `name` property. Three concrete agents are provided:

- **RandomAgent:** Picks uniformly at random from the legal move set. It serves as the lower-bound baseline.
- **LearningAgent:** Wraps any trained `ChessModel` and its corresponding encoder. At inference time, it encodes the board, runs a forward pass, masks illegal moves by setting their logits to $-\infty$, and picks the move via greedy argmax or temperature-sampled argmax with optional top-k filtering.
- **UCIAgent:** Communicates with any UCI-compatible engine (e.g., Stockfish) via `python-chess`'s engine protocol. It supports configurable depth, time limits, skill levels, and MultiPV analysis for obtaining centipawn-scored move distributions.

3.2.3 Training and Evaluation Engine

The training subsystem centres on the `Trainer` class, which runs the supervised training loop.

A loss factory picks the appropriate loss function based on the head type. `PolicyLoss` computes KL divergence against cross-entropy against hard labels with optional label smoothing. `ValueLoss` uses mean squared error between the predicted tanh-bounded value and the target. `DualLoss` combines both with configurable weights.

The trainer wraps the forward pass in `torch.autocast` (float16 on CUDA/MPS) and applies `GradScaler` on CUDA to avoid underflow in low-precision gradients. Non-GNN backbones are passed through `torch.compile` for graph-level kernel fusion and optimised execution. Two learning rate schedules are supported — cosine annealing and step decay — both configured via YAML. Model weights, optimiser state, scheduler state, epoch counter, and best validation loss are serialised to disk after every epoch. Intermediate checkpoints are pruned to keep only every fifth epoch, the best, and the final model. As training progressed, the initial streaming Parquet pipeline was superseded by a multi-stage data preparation system. This pipeline is described in Section 3.3.8.

3.3 Design Decisions and Justification

3.3.1 Choice of Python and PyTorch

Python was chosen for its dominance in the machine learning ecosystem and the maturity of supporting libraries (`python-chess`, `pyarrow`, `torch_geometric`). PyTorch was preferred over TensorFlow for its eager-mode debugging, first-class support for dynamic computation graphs — essential for the variable-length PieceTransformer and graph-structured GNN inputs — and its `torch.compile` JIT pathway for production-grade throughput.

3.3.2 Backbone-Head Decoupling

Backbone feature extraction is deliberately separated from task-specific output heads. Every backbone subclasses `ChessModel` and implements `forward_backbone()` to produce a fixed-dimension feature vector; the `set_head()` method then attaches one of three interchangeable heads. This decoupling yields $6 \times 3 = 18$ valid backbone-head combinations from six backbone and three head implementations, without combinatorial code explosion. More importantly, the only thing that changes when comparing architectures is the backbone itself - the output layer, loss function, and training loop stay identical.

3.3.3 Streaming Data Pipeline

The dataset is stored in Apache Parquet format, which supports columnar compression and row-group-level random access. The `ChessDataset` class, implemented as a PyTorch `IterableDataset`, reads one row-group at a time and shuffles within each group. Peak memory is bounded by the size of a single row-group, regardless of the total dataset size - a major constraint when scaling to hundreds of millions of positions. Multi-worker data loading is supported by partitioning row-groups across workers using their `worker_id`.

3.3.4 Move Encoding

Rather than the $8 \times 8 \times 73$ spatial policy planes used by AlphaZero, the framework uses a flat vector of 4544 logits. Each index corresponds to a unique UCI move string, generated from all valid (from_square, to_square, promotion) tuples. This representation is architecture-agnostic: the same policy head works for CNNs, Transformers, and GNNs without any modification. The trade-off is losing spatial locality in the policy output, which the flat head compensates for with a hidden layer.

3.3.5 Dual-Headed AlphaZero-Style Architecture

The default configuration uses a dual-headed model that jointly predicts the policy distribution and the board evaluation from shared backbone features. This follows the AlphaZero paradigm: the value head gives the backbone a training signal for positional understanding, while the policy head learns to select tactical moves. The combined loss $\mathcal{L} = \lambda_p \cdot \mathcal{L}_{\text{policy}} + \lambda_v \cdot \mathcal{L}_{\text{value}}$ is weighted equally by default ($\lambda_p = \lambda_v = 1.0$).

3.3.6 Spatial Policy Head

The initial policy head applied global average pooling to the backbone's spatial feature maps before projecting to the 4544-move logit vector. That discards square-level information critical for move selection — which piece on which square should move where. The spatial policy head replaces global average pooling with a 1×1 convolution that reduces the channel dimension, followed by a flatten operation that preserves the 8×8 spatial grid. The resulting $8 \times 8 \times C'$ representation is then projected to move logits via a linear layer, letting the network associate specific board locations with specific moves.

3.3.7 Squeeze-and-Excitation Blocks

Squeeze-and-Excitation (SE) blocks were added to the ResNet backbone's residual blocks. Each SE block applies global average pooling to "squeeze" the spatial dimensions into a channel descriptor, passes it through a two-layer fully connected bottleneck with ReLU activation, and uses the resulting channel weights to "excite" (scale) the original feature maps via element-wise multiplication. This channel attention mechanism lets the ResNet dynamically prioritise different feature channels depending on position — emphasising pawn-structure planes in endgames, say, and King-safety planes in middlegames. The SE blocks sit within the residual blocks, so skip connections maintain gradient stability throughout.

3.3.8 Data Pipeline Evolution

The initial streaming Parquet pipeline worked well for prototyping but became a bottleneck at scale: board encoding during runtime via `python-chess` was CPU-bound and couldn't saturate the GPU during training. A three-stage offline pipeline replaced it:

1. **Filtering (DuckDB):** Dataset entries are filtered based on the depth of the line generated by stockfish and the depth of the stockfish that generated the same. Anything below depth 18 was dropped, and with centipawn evaluation of greater $\pm$ 4000 was discarded as well.

2. **Deduplication (DuckDB):** FEN strings are normalised (stripping move counters) and grouped using DuckDB's out-of-core SQL engine. When duplicate positions exist, the analysis with the highest depth or node count is kept.

3. **PV Unrolling:** For each base position, the first N moves of the principal variation are played out. Each intermediate position inherits the root evaluation (with sign flips for alternating sides), avoiding the cost of re-evaluating with Stockfish. Drift analysis confirmed that 96% of inherited evaluations remain within $\pm$100 centipawns of the true value.

4. **Deduplication (DuckDB):** Positions are deduplicated again since due to PV unroll it might be possible to arrive at the same board position, in which case the position with the highest depth or node count is kept and rest discarded.

5. **Target Computation:** We store the best move for the particular chess board position and encode the centipawn position to the value to be used during training to speed up data loading.

The resulting dataset occupies 68.83 GB for 1.8 billion positions. It loads via a `torch.utils.data.DataLoader` read from a parquet file, `persistent_workers`, and configurable `prefetch_factor`.

3.4 Chess Implementation

Chess rules aren't reimplemented; the framework delegates all legality checking, move generation, and game-over detection to the `python-chess` library. This keeps the codebase focused on machine learning concerns while relying on `python-chess` for correctness in edge cases like en passant, castling through check, under-promotion, and the fifty-move rule.

Board representation is determined by the encoder (Section 3.2.1) rather than by a single canonical format. Each model family therefore receives the board in its most natural form: a spatial tensor for CNNs, a graph

for GNNs, and a token sequence for Transformers.

3.5 Web Platform Design

3.5.1 Platform Requirements

The web platform provides a browser-based interface for interacting with trained models. Its core requirements are:

- A user must be able to play a full game of chess against any trained model (Human-vs-Bot mode).
- A user must be able to select two models and watch them play against each other in real time (Bot-vs-Bot mode).
- The interface must support selecting from all available checkpoints.

3.5.2 System Architecture

The platform follows a client-server architecture. The backend is written in Python, chosen for its ease of use with the ML models and deployment. The frontend uses Svelte, a compile-time reactive framework that produces minimal JavaScript bundles and efficient DOM updates - well-suited to the frequent, localised state changes involved in rendering a chessboard. The frontend and backend communicate over a REST API which also handles session management, model listing, and game creation.

3.5.3 Game Modes

The platform supports three game modes. In **Human-vs-Bot** mode, the user makes moves through the board UI; each move is sent to the backend, which invokes the selected model's `get_move` method and fetches the response back. In **Bot-vs-Bot** mode, the backend orchestrates alternating `get_move` calls between two selected models, broadcasting each move to connected spectators with a delay for readability. In **Human-vs-Human** mode, the user(s) can make moves through the board UI.

3.5.4 User Interface Design

The interface centres on an interactive chessboard rendered using an SVG-based board component. Input is handled via drag-and-drop, with legal-move highlighting. A side panel displays the move list in standard

algebraic notation, a real-time evaluation bar (using a stockfish judge), best move highlighting, and model selection controls.

3.5.5 Backend API Design

The backend exposes the following core endpoints:

- `GET /api/runs` - List available model checkpoints.
- `POST /api/game/new` - Create a new game session, specifying mode and model(s).
- `GET /api/game/{id}` - Endpoint for querying the game state.
- `POST /api/game/{id}/move` - Submit a human move (Human-vs-Bot mode/Human-vs-Human mode).
- `POST /api/game/{id}/bot_move` - Submit a bot move (Human-vs-Bot mode/Bot-vs-Bot mode).
- `GET /api/game/{id}/eval` - Get the current evaluation of the game based on a stockfish judge.
- `POST /api/game/{id}/resign` - Resign the current game (Human-vs-Bot mode/Human-vs-Human mode).

The backend loads PyTorch models communicating board states and receiving moves over REST API.

3.6 Technical Challenges

Several technical challenges came up during design and were addressed through targeted engineering decisions. GNN inputs can't be trivially stacked along a batch dimension because each graph has a variable number of edges. The collation function handles this by offsetting edge indices by $i \times 64$ for the i-th graph in the batch and concatenating all edges into a single $(2, \sum E_i)$ tensor, with a `batch` vector mapping each node to its graph. This follows the PyTorch Geometric batching convention.

The PieceTransformer presents a different batching problem: its sequences range from 1 to 32 tokens depending on piece count. A key-padding mask, constructed from the `attention_mask` field, stops padded positions from influencing attention computation.

Mixed-precision training required platform-specific handling. The MPS backend supports float16 autocast but not `GradScaler`. So the trainer enables the scaler only on CUDA devices while still using autocast on MPS for throughput gains.

Chapter 4

Implementation

4.1 Development Environment and Tools

The framework is implemented in Python 3.10 with PyTorch as the deep learning backend. Table 4.1 lists the principal libraries and what they do.

Table 4.1: Key libraries and their roles in the framework.

Library	**Purpose**
PyTorch	Neural network definition, training, and inference
PyTorch Geometric	Graph convolution and attention layers (GCN, GAT)
python-chess	Board representation, move generation, engine protocol
PyArrow	Parquet file I/O for the training dataset
NumPy / Pandas	Numerical operations and data analysis
Matplotlib	Evaluation flow charts and training curve visualisation
PyYAML	Configuration file parsing
tqdm	Progress bars for training
pydantic-settings	Parsing of config yaml

All experiments ran on two hardware configurations: an Apple M-series workstation using the MPS (Metal Performance Shaders) backend for local development, and an NVIDIA GPU (CUDA) for main training. The codebase auto-detects the best available device at startup — preferring CUDA to MPS over CPU — and enables hardware-specific optimisations (TF32 math and cuDNN auto-tuning on CUDA).

Version control uses Git. All hyperparameters live in a single YAML configuration file (`config/settings.yaml`), parsed at runtime into a hierarchy of Python dataclasses (`Config`, `ModelConfig`, `TrainingConfig`, etc.), so every experiment is fully reproducible from its configuration snapshot.

4.2 Core Framework Implementation

4.2.1 Abstract Base Classes

The framework's extensibility rests on two abstract base classes:

- `ChessModel(ABC, nn.Module)`: Every neural network backbone subclasses this and implements `forward_backbon` (returning a fixed-dimension feature vector) and `get_backbone_output_dim()`. The base class provides a concrete `forward(x)` that chains the backbone with whichever output head has been attached via `set_head()`.
- `StateEncoder(ABC)`: Every board encoder implements `encode(board)` and `get_input_shape()`. This lets the data pipeline and agents work with any encoder-model pairing without type-checking.

4.2.2 Factory Pattern

The `create_model(config)` factory reads the backbone name and head type from the configuration, instantiates the appropriate backbone with its architecture-specific parameters (e.g., number of residual blocks, number of attention heads), constructs the matching output head, and attaches it. A companion `get_encoder_for_model(backbone)` factory maps backbone names to their corresponding encoder classes. This two-step factory keeps backbone-encoder pairings consistent.

4.2.3 Configuration System

All tunable parameters are centralised in a YAML file and parsed into nested dataclasses:

- `HardwareConfig`: Device selection and data-loader worker count.
- `ModelConfig`: Backbone type, head type, and per-family sub-configs (`CNNConfig`, `TransformerConfig`, `GNNConfig`).
- `TrainingConfig`: Batch size, learning rate, weight decay, epoch count, loss weights, and LR scheduler type.
- `EngineConfig`: Path to the Stockfish binary and default search parameters.

Pydantic-settings parses the yaml file and allows us to access the config options anywhere in the project.

4.3 Supervised Learning Model

4.3.1 Data Preparation

Training data comes from the Lichess Stockfish evaluation dataset, which provides millions of positions annotated with engine analysis. The data is stored in Apache Parquet format for efficient columnar compression and row-group-level random access.

The `ChessDataset` class, implemented as a PyTorch `IterableDataset`, the following Parquet schema:

1. **PV-unrolled:** Each row contains a FEN string, the best move in UCI notation, a precomputed value. It is structured as f, b, v which was artifact of converting jsonl to parquet when doing the data-cleanup/preparation. Where the fen represents the current board state and the b is the best move from the board state and v is a value from +1 to -1 depending on the side that is currently winning.

Data loading works in following steps. First, the dataset lists all row-groups across the Parquet file. These are partitioned across DataLoader workers by worker ID. Finally, each row is converted to a training example by encoding the board and building the policy and value targets.

Policy targets are constructed in *hard label* mode, the best move receives a probability of 1.0 whereas everything else is set to 0 in the super sparse 4544-dimensional vector.

Value targets are taken from the dataset: the game result ($+1$, 0, -1), the mate score (mapped to ± 1), or the centipawn evaluation (divided by 1000 and clamped to $[-1, 1]$). The value is always expressed based on the colour that is winning.

A custom `collate_fn` batches examples into the format expected by each model family. CNN inputs are stacked along the batch dimension. Transformer inputs have each dictionary field (tokens, positions, masks) stacked independently. GNN inputs require more care: edge indices are concatenated with per-graph offsets of 64 nodes, and a `batch` vector maps each node to its originating graph, following the PyTorch Geometric batching convention.

4.3.2 Neural Network Architecture

Six backbone architectures are implemented, two per model family.

4.3.2.1 Convolutional Neural Networks

ConvNet consists of six sequential convolutional blocks, each comprising a 3×3 convolution (with padding to preserve spatial dimensions), batch normalisation, and ReLU activation. The first block projects the 18-channel input to 256 channels; subsequent blocks maintain this width. A global average pooling layer reduces the $256 \times 8 \times 8$ feature maps to a 256-dimensional vector.

ResNet begins with an initial 3×3 convolution that projects the input to 256 channels, followed by N residual blocks (configurable to 6, 10, or 20). Each residual block contains two 3×3 convolutions with batch normalisation and a SE block, and the input is added to the output via a skip connection before the final SiLU. Global average pooling produces the output vector.

4.3.2.2 Transformers

SquareTransformer tokenises the board as 64 fixed tokens (one per square) using a vocabulary of 13 (empty + 12 piece types). Each token is embedded and summed with a learnable positional embedding and a coordinate projection (normalised rank-and-file). Side-to-move information is broadcast as a bias to all tokens, while castling rights are injected into a prepended CLS token. The sequence passes through N Transformer encoder blocks, each using pre-LayerNorm, multi-head self-attention (8 heads), and a GELU-activated feed-forward network with a $4\times$ expansion ratio. The CLS token's final hidden state serves as the backbone output.

PieceTransformer tokenises the board as a variable-length sequence of up to 32 tokens (one per piece). Each token encodes the piece type (vocabulary of 12) and its square position via separate embedding tables. A key-padding mask prevents attention from flowing to padded positions. A CLS token aggregates the sequence representation.

4.3.2.3 Graph Neural Networks

GCN projects the 14-dimensional node features to a hidden dimension of 256 via a linear layer, then applies N GCN layers. Each layer performs a graph convolution (`GCNConv`), batch normalisation, and ReLU activation with a residual connection. Global mean pooling aggregates node embeddings into a single graph-level vector, which is summed with a side-to-move embedding and a castling projection before a final linear output layer.

GAT follows the identical structure but replaces `GCNConv` with `GATConv`, which computes multi-head attention (4 heads) over neighbours. The GAT layers accept the 3-dimensional edge attributes (distance, ray-tracing, legality) via the `edge_dim` parameter, allowing the attention mechanism to weight edges by their tactical significance. ELU activation replaces ReLU for smoother gradients in the attention computation.

4.3.3 Training Process

4.3.3.1 Loss Functions

Three loss modules are provided:

- **PolicyLoss:** Cross-entropy for hard labels, with optional label smoothing.
- **ValueLoss:** Mean squared error between the predicted value (tanh-bounded) and the target.
- **DualLoss:** Weighted sum $\mathcal{L} = \lambda_p \mathcal{L}_{\text{policy}} + \lambda_v \mathcal{L}_{\text{value}}$.

4.3.3.2 Optimisation

Training uses the AdamW optimiser with a default learning rate of 10^{-3} and weight decay of 10^{-4}. Learning rate scheduling supports cosine annealing (default) and step decay. Gradients are clipped to a maximum norm of 1.0 to prevent instability during early training.

4.3.3.3 Mixed Precision and Compilation

On CUDA devices, training uses float16 autocast with `GradScaler` to maintain numerical stability. On MPS (Apple Silicon), autocast is enabled but the scaler is disabled — MPS doesn't support it. Non-GNN backbones are wrapped with `torch.compile` for kernel fusion and operator-level optimisation. GNN models had to be excluded from compilation because their dynamic graph structures prevent effective tracing.

4.3.3.4 Checkpointing

After each epoch, the trainer serialises the model state dict, the optimiser state, the scheduler state, the epoch number, the global step count, and the best validation loss. At the end of training, intermediate checkpoints are pruned to keep only every fifth epoch, reducing storage overhead while preserving enough granularity for post-hoc analysis.

4.4 Architectural Enhancements

After the initial implementation and early training runs, several architectural improvements addressed weaknesses I'd noticed.

4.4.1 Spatial Policy Head

The policy head uses spatial features when available: a 1×1 convolution reduces channels to C', batch norm + ReLU is applied, then the 8×8 grid is flattened to a $64C'$ vector and mapped by a linear layer to 4544 logits. If spatial features are not available, it falls back to a two-layer MLP on the backbone vector.

4.4.2 Squeeze-and-Excitation Blocks

SE blocks were inserted into each `ResidualBlock`. After the second convolution and batch normalisation, the feature maps are squeezed via global average pooling to a channel vector of dimension C, passed through a bottleneck (`Linear(`C`,` C/r`)` $\rightarrow$ ReLU $\rightarrow$ `Linear(`C/r`,` C`)` $\rightarrow$ Sigmoid, with reduction ratio $r = 16$), and the resulting channel weights multiply the original feature maps. The SE block sits inside the residual path, so the skip connection ensures stable gradient flow even in deep (20-block) configurations.

4.4.3 Activation Functions

ReLU activations in the ConvNet and ResNet backbones were replaced with SiLU (Swish), which gives smoother gradients near zero. The Transformer blocks already used GELU, so no change was needed there.

4.5 Data Pipeline Evolution

The initial streaming Parquet pipeline (Section 4.5.1) worked well for prototyping but became the training bottleneck at scale.

4.5.1 Stage 1: Filtering and Deduplication

DuckDB's out-of-core SQL engine normalises FEN strings (stripping halfmove and fullmove counters), groups by the normalised FEN, and keeps only the row with the highest analysis depth or node count. This eliminates redundant positions that would otherwise bias the training distribution toward common openings. Also removes any entries that didn't meet the criteria of >= 18 depth or had a centipawn evaluation that fell outside the range of $\pm$ 4000.

4.5.2 Stage 2: PV Line Unrolling

For each base position, the first N moves of the principal variation are played out using `python-chess`. Each intermediate position is emitted as a separate training example, with the subsequent PV move as its policy

target. The root evaluation inherits sign flips across alternating sides — avoiding the prohibitive cost of re-evaluating every unrolled position with Stockfish. A drift analysis validated this: 96% of inherited evaluations stayed within ±100 centipawns of the independently computed value.

4.5.3 Stage 3: Deduplication and Target Computation

Positions were normalised again since during the PV unrolling we could have arrived at the same position, in which case the position with the higher depth count or knodes count was kept and others were discarded. By doing this we were able to go from about 3 billion positions to 1.8 billion positions. Engine centipawn scores are converted value targets as ± 1 which are stored as `float16` (2 bytes).

4.6 Baseline Models

Two baselines anchor the evaluation:

- **RandomAgent:** Plays uniformly at random from the legal move set. It's the lower bound — if a trained model can't beat random, something has gone wrong.
- **UCIAgent (Stockfish):** Wraps Stockfish at configurable depth and skill levels. At full strength (depth 15+), Stockfish sets the upper bound. At reduced depths (1-5), it provides intermediate calibration points for estimating Elo.

4.7 Benchmarking and Evaluation Pipeline

The benchmarking system automates model evaluation through six stages:

1. Each trained model plays a configurable number of games against Stockfish, alternating colours for fairness.
2. After every move, Stockfish analyses the resulting position at depth 18, recording the centipawn score from White's perspective. Mate scores are converted to large constants (±10000).
3. Each game is exported in Portable Game Notation with embedded evaluation comments, enabling review in standard chess software (Lichess, Chess.com, ChessBase).
4. Matplotlib generates evaluation flow charts for each game, plotting centipawn score against move number with win/draw/loss shading. A combined comparison plot overlays all models.

5. A Markdown report is generated with win/draw/loss tables, timing statistics (mean, median, min, max), and links to PGN and figure files.

4.8 Documentation

The codebase follows a consistent documentation standard: every module, class, and public method has a docstring specifying its purpose, arguments, and return values. A `README.md` covers installation, a quick-start guide, and example training commands. The `setup.sh` script automates environment setup and dependency installation.

Chapter 5

Evaluation

5.1 Evaluation Methodology

This evaluation combines tactical depth using puzzles and playing elo using stockfish to provide a balanced assessment of each architecture. The benchmarking pipeline is automated via the project scripts so that training runs can be compared without manual intervention.

Benchmarking Pipeline: Models are evaluated through bot-vs-bot round-robins and fixed-depth Stockfish matches. **Metrics Defined:**

- **Estimated Elo:** Derived from match outcomes aggregated across repeated runs against Stockfish and baseline models.
- **Average Centipawn Loss (ACPL):** Evaluates the objective quality of individual moves compared to a high-depth Stockfish evaluation.
- **Win/Draw/Loss (WDL) Ratios:** Analyzes the playstyle stability and risk profile of each model.
- **Puzzle Solving Accuracy:** Computed on held-out positions from the puzzles dataset to measure tactical depth.

Experimental Scope: Six architectures (ConvNet, ResNet, GAT, GCN, Piece Transformer, Square Transformer) were evaluated across multiple dataset scaling checkpoints (10M, 100M, 200M, 500M, and 1000M parameters/games).

5.2 Scaling and Data Efficiency

To understand how dataset size impacts performance, score percentages were analyzed across different dataset scales.

Across all backbones, models generally improved as data scaled from 10M to 1000M games. ResNet and Transformer architectures scaled efficiently, demonstrating significant performance gains with larger datasets. In contrast, Graph Neural Networks (GNNs), such as GAT and GCN, struggled to scale and their performance flatlined at an estimated Elo of 800 regardless of the data size, highlighting fundamental difficulties in their current batching and message passing implementations.

5.3 Playing Strength & Match Outcomes

When evaluated against fixed-depth Stockfish and baseline agents, the strongest supervised models consistently out-performed baselines and demonstrated clear Elo progression.

Estimated Elo Ratings: ResNet achieved the highest peak playing strength, reaching an estimated 2498 Elo at the 200M scale. ConvNet and Square Transformers also showed strong baseline performance, comfortably exceeding 2100 Elo.

Win/Draw/Loss Profiles: The WDL breakdown exposes the risk profiles of the models. Some models exhibit high draw rates, indicative of conservative but safe play, while others display high win/loss variance, suggesting an aggressive approach that may overfit to tactical motifs but occasionally blunders.

Elo Logistic Fits: Logistic fit curves validate the mathematical confidence of the Elo estimations, confirming that the recorded win probabilities scale logically against expected opponent strength.

5.4 Tactical Accuracy & Move Quality

Beyond match outcomes, the objective quality of individual moves provides insight into the models' positional understanding and tactical awareness.

Average Centipawn Loss (ACPL): ACPL metrics reveal how closely the models mirror Stockfish's evaluations. A tighter ACPL spread indicates a lower frequency of blunders. The strongest CNN and Transformer models maintain smooth evaluation curves, indicating consistent move quality.

Puzzle Benchmark Results: Evaluation on held-out puzzle positions demonstrates the models' ability to find forced tactical continuations. High-capacity models convert material advantages more reliably and avoid

obvious tactical blunders that plague the baseline agents.

Evaluation Trajectories: Cumulative evaluation trajectory graphs illustrate how games unfold. Stronger models are able to maintain and smoothly convert advantages, while weaker models or smaller data scales are characterized by abrupt evaluation swings indicative of late-game blunders.

5.5 Comparative Architectural Analysis

Synthesizing the results reveals distinct trade-offs between the evaluated backbones:

- **CNNs (ResNet & ConvNet):** Provide the strongest baseline performance and highest training efficiency. ResNet achieved the highest peak Elo (2498), proving to be robust and highly effective for this dataset scale.
- **Transformers (Square & Piece):** Deliver strong global pattern recognition and flexible feature extraction. The Square Transformer scaled well (peaking around 2134 Elo), but these models come with heavier computational costs and moderate overhead due to attention operations.
- **GNNs (GAT & GCN):** While structurally aligned with piece relationships, they severely underperformed in practice. They were limited by batching overheads, smaller effective batch sizes, and flatlined early in training.

5.6 Web Platform Evaluation

5.6.1 Usability Testing

The web platform was evaluated through structured walkthroughs of the human-vs-bot and bot-vs-bot workflows. Users reported that game setup and model selection were straightforward, with the configuration panels providing sufficient guidance for non-expert users. The human-vs-bot mode was rated as the most intuitive. Bot-vs-bot mode required slightly more explanation due to the additional parameters but remained usable after brief instruction.

5.6.2 Performance and Responsiveness

Move latency is dominated by model inference and varies by architecture. CNNs return moves quickly enough for real-time play, while transformer and GNN models introduce small but noticeable delays under higher

depth settings. The backend remained responsive for small concurrent loads, with websocket updates keeping the board state consistent.

5.6.3 Platform Match Validations

Web-based matches were cross-checked against offline benchmarks. Results are consistent within expected variance, indicating that the platform does not introduce systematic bias. Minor discrepancies are attributed to stochastic opening selection and batching order during inference.

5.7 Discussion & Limitations

Compared with published chess models, these results align with expectations for a single-machine supervised pipeline. The models sit between classical heuristic engines and large-scale reinforcement learning systems, offering stronger tactical play than shallow baselines while remaining computationally accessible. The findings validate the framework's backbone-head decoupling and reinforce the importance of consistent evaluation pipelines when comparing architectures.

Limitations: Evaluation was constrained by compute budgets and the depth of the Stockfish opponents used for labeling and testing, which restricts the ability to measure strength against top-tier engines. Furthermore, the GNN implementations introduced significant engineering complexity and batching overheads that ultimately throttled their learning capacity. Future work could address these architectural bottlenecks and incorporate full reinforcement learning pipelines.

Chapter 6

Critical Review and Reflection

6.1 Achievement of Objectives

The primary objective of this project was to engineer and evaluate a suite of deep learning models capable of playing chess without search algorithms. The framework implements all six proposed architectures (ConvNet, ResNet, SquareTransformer, PieceTransformer, GCN, GAT), each trainable under identical data conditions with interchangeable output heads. Both the modular backbone-head design and the streaming data pipeline.

The secondary objective of cross-architecture comparison are supported by the benchmarking pipeline, which records and exports PGN files for qualitative analysis.

Beyond the initial architecture, the framework evolved during development. The ResNet backbone was augmented with Squeeze-and-Excitation blocks for dynamic channel attention. A spatial policy head replaced global average pooling to preserve square-level information. The data pipeline was also designed to be capable of handling over one billion positions.

6.2 What Worked Well

The abstract `ChessModel` interface made adding new architectures straightforward. Adding the GAT architecture, for instance, meant implementing a single class with two methods; the training loop, loss functions, and evaluation pipeline needed zero modifications. That backbone-head decoupling justified the upfront design effort many times over.

Row-group-level streaming from Parquet files enabled training on datasets larger than available RAM without changing the training loop. This mattered most when scaling from initial prototyping (100K positions) to full-scale runs (tens of millions of positions). Centralising all hyperparameters in a single YAML file eliminated hard-

coded constants and made experiment management straightforward. Combined with checkpoint serialisation, any past experiment can be exactly reproduced.

The decision to inherit root evaluations (with sign flips) for PV-unrolled positions rather than re-evaluating each with Stockfish was both practical and accurate. The drift analysis, showing 96% of inherited values within ±100 cp, validated this and enabled a large increase in training data diversity at negligible computational cost.

6.3 What Didn't Work Well

The GNN encoder is much slower than the CNN encoder because it must compute attack and defence relations for every square on every board. This makes the GNN data pipeline the bottleneck during training, even though the model itself is no larger than the CNN.

The original policy head used global average pooling, discarding spatial information critical for move prediction. This was caught early and replaced with a spatial policy head, but those initial training runs on the pooled representation produced noticeably weaker policy accuracy — confirming that square-level information is essential for move selection.

6.4 Lessons Learned

Filtering out low-depth evaluations and balancing the value distribution had a larger impact on model performance than simply increasing dataset size. Data quality consistently mattered more than data quantity.

The choice of board representation (spatial tensor vs graph vs token sequence) determines what information is accessible to the model and what must be learned from scratch. Encoder design proved as consequential as model architecture.

The ConvNet, despite being the simplest model, was an invaluable debugging tool. Once it was training correctly, issues in more complex architectures (Transformer attention masks, GNN edge batching) could be diagnosed by comparison. Starting with the simplest architecture saved a lot of time.

Mixed-precision training requires platform-specific handling. The asymmetry between CUDA and MPS support for `GradScaler` required non-obvious conditional logic that initially caused silent numerical instability.

6.5 Personal Reflection

This project deepened my understanding of deep learning engineering well beyond what I covered in lectures. Implementing graph neural networks from scratch — including the non-trivial batching of variable-edge graphs — required engaging with PyTorch Geometric at a level far beyond its tutorial examples. The Transformer implementations reinforced how much positional encoding design matters: absolute, relative, and learnable encodings produced noticeably different training dynamics on the same data.

Managing a multi-architecture codebase also developed my software engineering discipline. The abstract base class pattern — which initially felt like over-engineering for a single-developer project — turned out to be essential when I added the sixth architecture three months into development.

6.6 Future Work

6.6.1 Reinforcement Learning

The framework currently relies entirely on supervised learning from engine-annotated data. Two reinforcement learning paradigms are natural extensions. Recording MCTS visit-count distributions as policy targets and game outcomes as value targets — enabling the model to improve beyond the ceiling of its training data. Alternatively, playing the model against Stockfish at progressively increasing difficulty (varying skill level and search depth) would give a structured reinforcement signal.

6.6.2 Web Platform

The planned Python/Svelte web platform would give users a browser-based interface for interacting with trained models. They could play any trained model via a drag-and-drop board interface, or watch two models play each other with spectator support, configurable move delay, and live evaluation bars. A round-robin tournament system — where all trained models play each other with automated Elo ratings calculated from the results — would allow systematic strength comparisons. An analysis board mode for setting up arbitrary positions and comparing move recommendations from different models side by side is also planned.

6.6.3 Long-term Research Directions

Framing chess as a sequence prediction task — where the model autoregressively predicts move continuations from a history of board states, analogous to language modelling — is one direction I find interesting. This Decision Transformer-style approach could enable genuine multi-move planning without explicit search. The

backbone–head architecture is also game-agnostic, so adapting the framework to other perfect-information games (e.g., Shogi, Othello) would test the generality of the design.

6.7 Broader Impact

The modular design lowers the barrier to entry for researchers who want to experiment with novel architectures or training regimes without reimplementing the full chess AI stack. The web platform, once complete, would serve as a demonstration tool for communicating deep learning concepts to non-specialist audiences.

Chapter 7

Legal, Social, Ethical, and Professional Issues

7.1 Ethical Considerations

7.1.1 Research Ethics

This project doesn't involve human participants, the collection of personal data, or experimentation on living subjects. Ethical approval was obtained through the University of Surrey's SAGE (Self-Assessment for Governance and Ethics) process, which confirmed the project falls within the low-risk category. All training data comes from publicly available chess game databases and open-source engine evaluations.

7.1.2 Responsible AI

The models developed here are narrow AI systems designed exclusively for chess move prediction and position evaluation, so they pose no direct risk of harm. That said, transparency in AI decision-making still matters. The framework's evaluation pipeline (centipawn tracking, PGN export) gives full traceability of every move decision, and the soft policy labels expose the model's uncertainty across candidate moves. Most commercial chess engines return only their top move without a probability distribution, so this level of interpretability is unusually high for the domain.

Bias in training data is worth noting: the Lichess Stockfish evaluation dataset reflects the engine's playing style and evaluation heuristics. Models trained on it will inherit Stockfish's strategic preferences — its material-focused evaluation, for instance — which may not represent alternative playing styles well. That's an acceptable limitation given the project's goal of distilling knowledge from a strong engine, not modelling diverse human play.

7.2 Legal Issues

7.2.1 Intellectual Property

The framework is original work for the University of Surrey Final Year Project and complies with the University's intellectual property policy. No proprietary code or datasets are used.

7.2.2 Copyright and Licensing

All third-party libraries are used in compliance with their respective open-source licences:

- `python-chess`: GPL-3.0 — permits use, modification, and distribution.
- PyTorch: BSD-3-Clause — permissive licence with minimal restrictions.
- PyTorch Geometric: MIT — permissive licence.
- Stockfish: GPL-3.0 — used as an external binary for evaluation; the framework does not link against or modify its source code.
- Lichess game data: Creative Commons CC0 — public domain, no restrictions on use.

The training dataset comes from Lichess's publicly available database, released under Creative Commons CC0 (public domain). The Stockfish evaluations annotating this data are produced by an open-source engine and carry no additional copyright restrictions.

7.3 Social Issues

7.3.1 Accessibility

The framework is implemented in Python and uses well-documented libraries, so the barrier to entry is relatively low for anyone with basic Python experience. The YAML configuration system and command-line training scripts help with this. The main accessibility limitation is hardware: training needs a GPU for practical turnaround times.

7.3.2 Web Platform Accessibility

The planned web platform targets modern evergreen browsers (Chrome, Firefox, Safari, Edge) and is designed to be responsive across desktop and tablet screen sizes. SVG-based board rendering ensures crisp display at

any resolution. Keyboard accessibility for move input and ARIA labels for screen readers are planned but not yet implemented.

7.3.3 Impact on Chess Community

The project contributes to the growing body of open-source research on chess AI. By providing a controlled multi-architecture comparison and an accessible web interface, it could serve as an educational resource for players curious about how neural networks evaluate positions. The models aren't strong enough to be useful for cheating in competitive play, and includes no mechanisms for real-time assistance during online games.

7.4 Professional Issues

7.4.1 Code of Conduct

Development follows the BCS (British Computer Society) Code of Conduct. The framework is designed as an open educational and research tool, which satisfies the public interest obligation. The codebase follows standard software engineering practices — version control, automated testing, documentation, and modular design — in line with the professional competence requirement. All results, including negative results and known limitations, are reported honestly throughout this dissertation.

7.4.2 Information Security

The framework processes no personal data and stores no user credentials. On the web platform side, the backend API validates all inputs (FEN strings, UCI move strings) before processing, and no sensitive information — API keys, passwords, or otherwise — is hard-coded in the repository.

7.4.3 Web Platform Security

The planned web platform addresses common web vulnerabilities in a few ways. All FEN strings and UCI moves received from clients are validated against `python-chess` before execution, preventing malformed input from reaching the model.

7.5 Sustainability

7.5.1 Environmental Impact

Neural network training is computationally intensive and carries a real carbon footprint. This project takes several steps to reduce that cost. Mixed-precision training halves memory bandwidth while increasing throughput by up to 2×, cutting wall-clock time and energy consumption per training run. The streaming data loader avoids redundant I/O by reading each row-group exactly once per epoch, and checkpoint pruning reduces storage overhead by keeping only every fifth epoch.

More fundamentally, search-less inference is the whole point of this project: a single forward pass consumes orders of magnitude less energy than the millions of node evaluations a searching engine performs.

7.5.2 UN Sustainable Development Goals

The SDGs most relevant to this project are SDG 4 (Quality Education), SDG 9 (Industry, Innovation, and Infrastructure), and SDG 12 (Responsible Consumption and Production). The connection to education is the most direct: the web platform lets players explore how different neural architectures evaluate the same position, making deep learning concepts tangible in a way that reading about tensor shapes doesn't. The cross-architecture comparison — CNN, Transformer, and GNN on identical data — is a modest contribution to AI research methodology (SDG 9); the real value lies in the controlled experimental setup rather than any single architectural breakthrough. The efficiency measures described above (mixed precision, search-less inference) are a small but genuine effort toward responsible resource use (SDG 12), particularly given how energy-intensive large-scale model training has become across the field.

7.6 Summary

This project uses open-source tools and public-domain data throughout, produces transparent outputs with full move traceability, processes no personal information, and takes concrete steps to reduce its computational footprint through mixed-precision training and search-less inference. The main ethical consideration — bias inherited from Stockfish's evaluation style — is documented and acceptable given the project's distillation goals. No major legal, social, or ethical risks were identified.

Chapter 8

Conclusion

This project investigated whether deep learning architectures can develop meaningful chess-playing ability without search algorithms. To find out, a modular framework was built that supports six neural network backbones (ConvNet, ResNet, SquareTransformer, PieceTransformer, GCN, and GAT), each trainable under identical conditions with interchangeable policy, value, and dual output heads.

The framework makes two main contributions. First, it provides a controlled, cross-architecture comparison of CNN, Transformer, and GNN models trained on the same Lichess Stockfish evaluation data with consistent preprocessing, loss functions, and evaluation protocols. Second, it shows that knowledge distillation from a strong engine via soft policy labels is a viable and scalable training signal for search-less models.

The project grew beyond its initial design. The ResNet backbone gained Squeeze-and-Excitation blocks for dynamic channel attention, and a spatial policy head replaced global average pooling to preserve the square-level information critical for move selection.

The data pipeline was redesigned into a system capable of handling over one billion positions. DuckDB-based deduplication removes redundant positions; PV line unrolling generates diverse intermediate positions with inherited evaluations (validated at 96% accuracy within ± 100 cp). The resulting dataset, with precalculated value target, enables efficient training at scale.

From a software engineering perspective, the backbone-head decoupling pattern, the factory-based model creation, and the YAML-driven configuration system collectively lower the barrier to experimentation: adding a new architecture means implementing a single abstract class, while all training, evaluation, and benchmarking infrastructure is reused without modification.

The project has real limitations. Without search, the models are expected to struggle with deep tactical sequences (the "horizon effect") and highly technical endgames. The GNN encoder's computational cost — driven by per-position attack relation computation — remains a training throughput bottleneck, despite the

offline pre-encoding pipeline addressing this for CNN and Transformer models.

Looking ahead, reinforcement learning via MCTS self-play and curriculum-based Stockfish RL is the clearest next step for pushing model strength beyond the supervised learning ceiling. The planned Python/Svelte web platform would provide an interface for human-vs-bot and bot-vs-bot interaction, and sequence modelling approaches (Decision Transformer-style autoregressive move prediction) could enable genuine multi-move planning without any explicit search.

In summary, this project shows that search-less chess is technically feasible and provides a useful testbed for comparing how different neural network families handle spatial, relational, and sequential structure. The framework — with its enhanced encoders, attention mechanisms, and billion-position data pipeline — provides the infrastructure for continued investigation.

Appendix A

Code Listings

Appendix B

Additional Results

Appendix C

User Guide

Appendix D

Game Examples

Appendix E

Ethical Approval Documentation

Appendix F

Project Planning Documents

Bibliography

[1] M. Campbell, A. J. H. Jr., and F. hsiung Hsu, "Deep blue." [Online]. Available: https://www.mimuw.edu.pl/~ewama/zsi/deepBlue.pdf

[2] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search." [Online]. Available: https://www.remi-coulom.fr/CG2006/CG2006.pdf

[3] F. Calderan, "Minimax chess engine." [Online]. Available: https://fvcalderan.github.io/myworks/articles/minimax_chess_engine.pdf

[4] M. C. Fu, "Monte carlo tree search: A tutorial." [Online]. Available: https://www.informs-sim.org/wsc18papers/includes/files/021.pdf

[5] G. DeepMind, "Mastering chess and shogi by self-play with a general reinforcement learning algorithm." [Online]. Available: https://arxiv.org/pdf/1712.01815

[6] E. Jenner, S. Kapur, V. Georgiev, C. Allen, S. Emmons, and S. Russell, "Evidence of learned look-ahead in a chess-playing neural network." [Online]. Available: https://arxiv.org/pdf/2406.00877

[7] E. O. David, N. S. Netanyahu, and L. Wolf, "Deepchess: End-to-end deep neural network for automatic learning in chess." [Online]. Available: https://arxiv.org/pdf/1711.09667

[8] M. Lai, "Giraffe: Using deep reinforcement learning to play chess." [Online]. Available: https://arxiv.org/pdf/1509.01549

[9] T. Rigaux and H. Kashima, "Enhancing chess reinforcement learning with graph representation." [Online]. Available: https://arxiv.org/pdf/2410.23753

[10] S. Alwer, "Graph neural networks for modelling chess." [Online]. Available: https://theses.liacs.nl/pdf/2022-2023-AlwerSaleh.pdf

[11] P. Hammersborg and I. Strümke, "Reinforcement learning in an adaptable chess environment for detecting human-understandable concepts." [Online]. Available: https://arxiv.org/pdf/2211.05500

[12] G. DeepMind, "Amortized planning with large-scale transformers: A case study on chess." [Online]. Available: https://arxiv.org/pdf/2402.04494

[13] N. Fiekas, "Python-chess: A chess library for python." [Online]. Available: https://python-chess.readthedocs.io/en/latest/

[14] C. P. Wiki, "Universal chess interface." [Online]. Available: https://www.chessprogramming.org/UCI

[15] Lichess, "Lila: The lichess chess server." [Online]. Available: https://github.com/lichess-org/lila

[16] C. P. Wiki, "Top chess engine championship." [Online]. Available: https://www.chessprogramming.org/TCEC

[17] Lichess, "Lichess api." [Online]. Available: https://github.com/lichess-org/api/blob/master/doc/specs/lichess-api.yaml

[18] IETF, "The websocket protocol." [Online]. Available: https://tools.ietf.org/html/rfc6455

[19] R. Harris, "Svelte: Cybernetically enhanced web apps." [Online]. Available: https://svelte.dev/

[20] D. Monroe and P. A. Chalmers, "Mastering chess with a transformer model." [Online]. Available: https://arxiv.org/pdf/2409.12272

[21] H. Baier and M. H. Winands, "Monte-carlo tree search and minimax hybrids." [Online]. Available: https://dke.maastrichtuniversity.nl/m.winands/documents/paper%2049.pdf

[22] Z. Tang, D. Jiao, R. McIlroy-Young, J. Kleinberg, and S. Sen, "Maia-2: A unified model for human-ai alignment in chess." [Online]. Available: https://arxiv.org/pdf/2409.20553